

Real-time Intrusion Detection System

Machine Learning Applied to Security

André Oliveira

Dep. of Electronics Telecommunications and Informatics
University of Aveiro
Aveiro, Portugal
andreaoliveira@ua.pt

Alexandre Cotorobai

Dep. of Electronics Telecommunications and Informatics
University of Aveiro
Aveiro, Portugal
alexandre.cotorobai@ua.pt

Abstract—Intrusion Detection Systems (IDSs) operating in real networks face continuously evolving traffic patterns, which can significantly degrade the performance of static, batch-trained models. This project presents a real-time IDS with incremental learning capabilities designed to adapt to concept drift while operating under realistic deployment constraints. The system is evaluated using the NF-CICIDS2018-v3 NetFlow dataset, with careful attention to data leakage prevention and class imbalance.

The proposed solution follows a two-stage architecture: an incremental binary classifier based on Stochastic Gradient Descent (SGD) performs real-time attack detection, while a Random Forest model identifies specific attack types. Online adaptation is enabled through a shadow-model strategy, where candidate updates are validated using the Matthews Correlation Coefficient (MCC) before deployment. Experimental results show that the incremental model improves steadily during streaming execution, increasing from an initial MCC of 0.64 (F1=0.76, Recall=0.66) to a final MCC of 0.78 (F1=0.87, Recall=0.94), demonstrating the effectiveness of online learning for adaptive intrusion detection.

Index Terms—Intrusion Detection Systems, Online Learning, Incremental Learning, Network Security, Concept Drift, Stream Processing, Machine Learning

I. INTRODUCTION

The steady expansion of networked infrastructures, from enterprise data centers to cloud platforms and Internet of Things (IoT) deployments, has increased both the attack surface and the operational complexity of defending modern networks. IDSs remain a fundamental security control, yet many traditional solutions rely on static signatures or models trained offline in a batch setting. In practice, network traffic is highly non-stationary: user behavior evolves, services change, and adversaries continuously adapt their tactics. Under these conditions, static IDS models tend to degrade over time, motivating the need for detection pipelines that can operate online and remain effective under distribution shifts.

This project addresses the problem of building an IDS that is simultaneously real-time and adaptive. Specifically, we target the challenge of maintaining reliable attack detection in streaming scenarios affected by concept drift, while avoiding disruptive full retraining cycles and limiting reliance on information that would be unavailable at inference time. These constraints are particularly relevant for practical deployments, where latency, stability, and resource usage must be balanced against detection performance.

To that end, we design and implement a real-time intrusion detection system with incremental learning. The system leverages machine learning models to (i) detect whether traffic is benign or malicious and (ii) identify the most likely attack category when an intrusion is detected. The solution is developed and evaluated using the NF-CICIDS2018-v3 NetFlow dataset [1], which provides large-scale, labeled NetFlow traffic spanning benign activity and multiple attack types. The methodology emphasizes deployment realism by mitigating data leakage, addressing severe class imbalance, and restricting the feature set to attributes that can be computed in an online setting.

Architecturally, the system adopts a two-stage pipeline: a lightweight incremental binary classifier provides fast filtering of benign traffic, and a higher-accuracy multi-class classifier assigns an attack type for detected intrusions. To safely update the online model, we integrate continuous performance monitoring and an automated retraining mechanism based on a shadow-model strategy, promoting updates only when they improve validation performance. A real-time dashboard complements the system by exposing detections, alerts, and model health indicators.

The remainder of this report is organized as follows. Section II reviews relevant work on intrusion detection and continual learning. Section III describes the dataset, data analysis, class balancing, preprocessing steps, feature engineering, dataset partitioning, and modeling choices. Section IV presents the system implementation and its main components. Section V reports the experimental evaluation and discusses security-relevant trade-offs. Finally, Section VI concludes the report and outlines directions for future work.

II. RELATED WORK

Continual and incremental learning have recently been explored as key enablers to keep IDSs effective in non-stationary environments, where new attacks and distribution shifts emerge over time. A key challenge in this setting is catastrophic forgetting, i.e., the degradation of performance on previously learned patterns after updating a model with new data. Talpini et al. [2] highlight that most machine-learning-based IDS solutions are still designed for static scenarios and tend to become obsolete as network traffic evolves. They

further stress that continual learning is particularly relevant in resource-constrained environments, such as edge or IoT deployments, where storing large amounts of historical traffic for retraining may not be feasible.

One prominent research direction explores hierarchical intrusion detection architectures combined with continual learning. In this context, Talpini et al. [2] propose a hierarchical IDS in which a Bayesian Neural Network (BNN) performs binary benign-versus-attack classification and is trained in a continual manner using variational inference, where the posterior distribution of a previous task is reused as the prior for subsequent tasks. For attack-type identification, the authors employ a generative classifier based on a diagonal approximation of Quadratic Discriminant Analysis (DQDA). This design allows new attack classes to be integrated incrementally, since class-conditional densities can be learned independently. Their evaluation on the CICIDS2017 dataset [3] shows that the continual BNN maintains stable binary detection performance without storing past data, while DQDA performs best when restricted to distinguishing among attack classes rather than including benign traffic.

A complementary line of work focuses on class-incremental learning (CIL) to enable IDS adaptation without full retraining, while also addressing privacy and deployment heterogeneity. Cerasuolo et al. [4] introduce a network-agnostic IDS in which attack patterns are learned centrally and later personalized by incrementally incorporating benign traffic from different network environments. This approach aims to avoid sharing sensitive local traffic data across domains. The authors evaluate several CIL strategies under both intra-dataset and cross-dataset scenarios, combining attack traffic from IoT-23 with benign traffic originating either from the same dataset or from Kitsune. To support early detection, they rely on packet-derived features extracted from the first ten packets of each biframe, including packet length, inter-arrival time, packet direction, and TCP window size.

From a methodological perspective, Cerasuolo et al. [4] analyze the trade-offs inherent to CIL, particularly between retaining knowledge of previously learned classes and effectively learning new ones. They discuss rehearsal-based approaches using memory buffers, regularization techniques such as knowledge distillation, and bias-correction strategies. Their experiments show that naive fine-tuning leads to severe forgetting, memory-based fine-tuning partially mitigates this effect, and distillation-based methods may suffer from strong intransigence when learning new classes. Moreover, their results indicate that adaptation across different network domains is significantly more challenging than adaptation within the same dataset.

Both studies [2], [4] highlight fundamental trade-offs in CIL for intrusion detection: retaining knowledge of previously learned classes versus effectively learning new ones. Rehearsal-based approaches using memory buffers can mitigate forgetting but require storing representative samples from past classes. Regularization techniques such as knowledge distillation preserve prior knowledge but may suffer from

intransigence when learning new classes. Talpini et al. [2] address this by eliminating the need for memory buffers through Bayesian continual learning, while Cerasuolo et al. [4] demonstrate that hybrid methods combining rehearsal and bias correction (e.g., Bias Correction (BiC)) can achieve near-optimal performance in intra-dataset scenarios.

Our work builds upon these insights but addresses a distinct problem: real-time intrusion detection with continual adaptation under concept drift using NetFlow-based features. Unlike Talpini et al. [2], who focus on Bayesian inference for continual learning, we adopt a simpler online learning approach using Stochastic Gradient Descent (SGD) with incremental updates via partial fitting, prioritizing computational efficiency and deployment simplicity. In contrast to Cerasuolo et al. [4], who evaluate early detection using the first ten packets, we work with complete NetFlow aggregations while systematically addressing data leakage by removing twelve temporal features that would be unavailable at inference time in streaming scenarios. Furthermore, we introduce a shadow-model deployment strategy to validate candidate models before promotion, ensuring safe updates in production environments, a critical consideration absent in prior work.

III. METHODOLOGY

This section describes the methodology adopted in the design of the proposed IDS, covering the dataset selection, data analysis, class balancing, preprocessing steps, feature engineering, dataset partitioning, and modeling choices. Particular emphasis is placed on ensuring deployment realism and compatibility with real-time streaming scenarios.

A. Dataset Description

The system is developed and evaluated using the NF-CICIDS2018-v3 dataset [1], a large-scale NetFlow-based dataset that captures realistic network traffic generated in a controlled environment. The dataset includes both benign activity and multiple categories of malicious traffic, such as distributed denial-of-service (DDoS), brute-force attacks, botnet activity, and infiltration attempts.

Each record corresponds to a network flow described by a set of numerical and categorical features extracted from packet-level information. In addition to a binary label indicating whether the traffic is benign or malicious, a multi-class label specifies the concrete attack type. This dual labeling enables both binary intrusion detection and fine-grained attack classification.

To make experimentation computationally feasible, a stratified sample of 100,000 flows is extracted from the original dataset while preserving the original class proportions.

B. Exploratory Data Analysis

1) *Feature Distributions*: An initial exploratory analysis is conducted to understand how individual features differ between benign and malicious traffic. Figure 1 illustrates the distributions of representative features, including TCP window

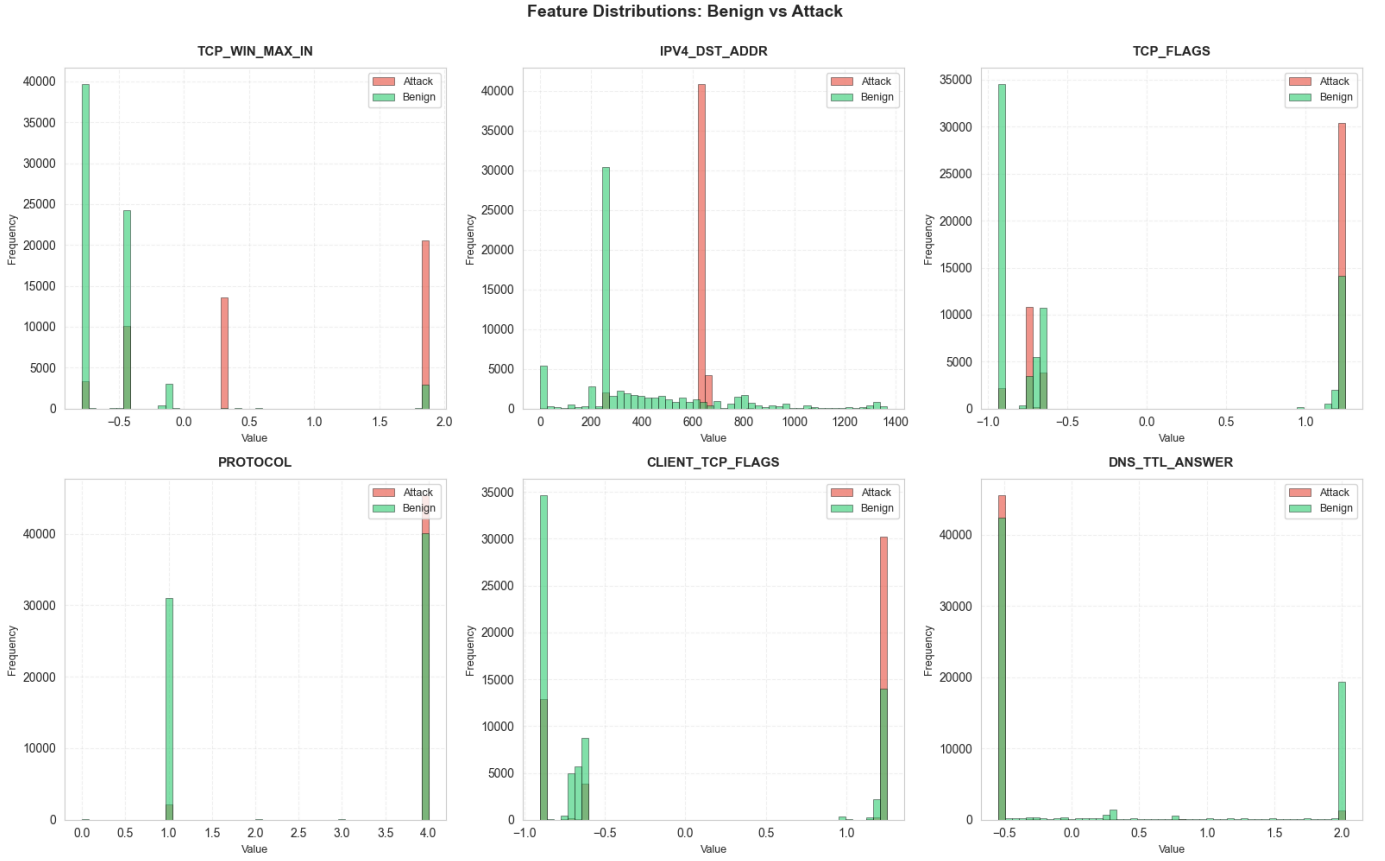


Fig. 1. Feature distributions comparing benign and attack traffic for selected NetFlow attributes.

sizes, destination IP addresses, protocol identifiers, and TCP flag combinations.

Several features exhibit clear separation between benign and attack traffic. For instance, TCP window-related features and TCP flag statistics show distinct patterns associated with automated attack tools and anomalous connection behavior. While some overlap remains unavoidable, these differences motivate the use of machine learning models capable of capturing complex decision boundaries.

2) *Label Distribution and Class Imbalance*: Figure 2 presents the binary and multi-class label distributions in the initial sampled dataset. Benign traffic accounts for approximately 88% of the samples, while malicious traffic represents only 12%, revealing a strong class imbalance.

At the attack-type level, several classes contain very few samples, which would severely hinder generalization. To mitigate this issue, attack categories with fewer than 500 instances are excluded from the study, ensuring sufficient data per class for meaningful learning.

C. Data Leakage Prevention

A critical design constraint of the system is its ability to operate in real time. Consequently, features that rely on post-hoc information or require knowledge of the full flow duration are removed to prevent data leakage. These include flow end

timestamps, flow duration metrics, and inter-arrival time (IAT) statistics computed over the entire flow.

Removing these features results in a more challenging learning problem but ensures that the reported performance realistically reflects what can be achieved in a streaming deployment, rather than exploiting information unavailable at inference time.

D. Class Balancing Strategy

To address the severe imbalance between benign and malicious traffic, an oversampling strategy is applied to the minority (attack) class. Rather than enforcing a fully balanced 50/50 split, a 60/40 benign-to-attack ratio is adopted to preserve a degree of realism while still improving attack recall.

Figure 3 illustrates the resulting label distributions after balancing.

Oversampling is performed with replacement while maintaining the relative proportions between attack types. This approach increases the representation of rare attacks without introducing synthetic samples that could distort the original data distribution.

E. Feature Correlation Analysis

To further understand the predictive relevance of individual features, a correlation analysis is performed between each

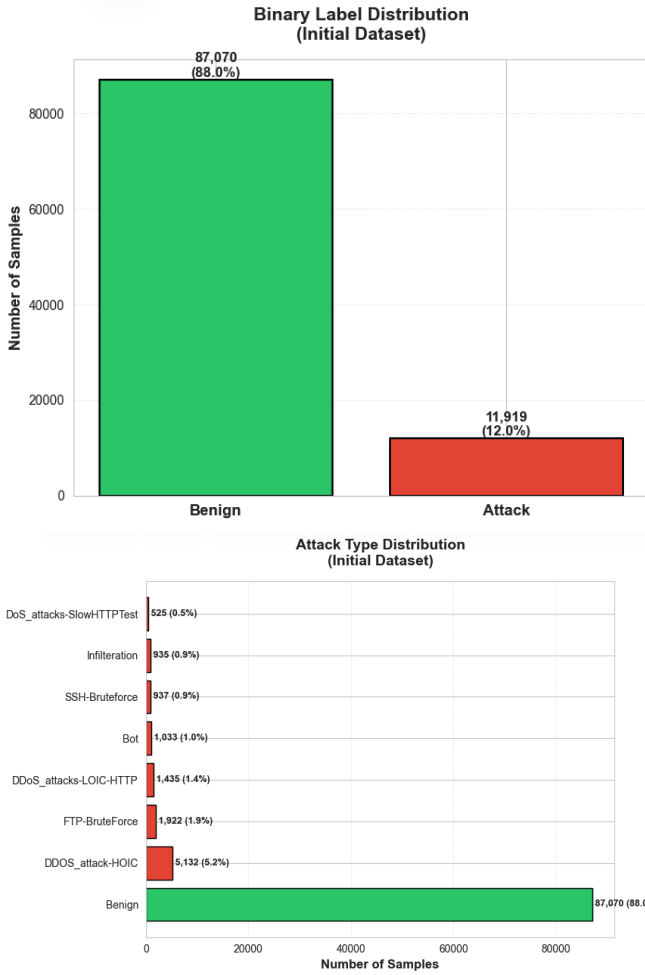


Fig. 2. Binary and multi-class label distributions before class balancing.

feature and the binary label. Figure 4 shows the correlation matrix of the top 15 most correlated features.

Features related to TCP window sizes, TCP flags, IPV4 destination addresses, and protocol identifiers show the strongest correlation with malicious activity. Although correlation alone does not imply causation, this analysis provides insight into which network characteristics are most informative for intrusion detection and supports the subsequent model selection.

F. Feature Extraction (NetFlow Representation)

Unlike packet-level approaches that extract features directly from raw Packet Capture (PCAP) payloads, this work operates at the flow level using the NF-CICIDS2018-v3 dataset. Each record corresponds to a NetFlow-style summary of a network communication flow, where traffic is represented through pre-computed statistical descriptors (e.g., byte and packet counts, rate-based measures, and protocol/application indicators). As a result, the *feature extraction* step is provided by the dataset generation pipeline itself: raw network activity is already transformed into fixed-length numerical vectors suitable for machine learning.

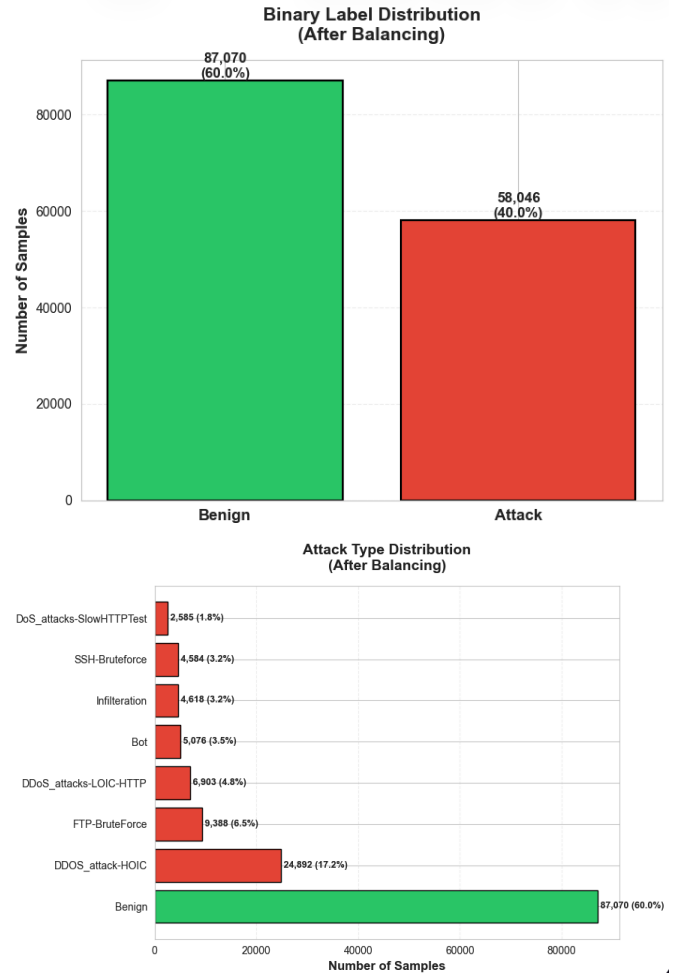


Fig. 3. Binary and multi-class label distributions after class balancing.

Our contribution focuses on selecting, sanitizing, and transforming these flow-level descriptors to ensure reliable training and evaluation in a realistic deployment setting. In particular, we remove fields that can lead to data leakage, standardize numeric variables, and encode categorical identifiers to obtain a consistent feature space for both offline training and online (streaming) inference.

G. Feature Engineering and Preprocessing

Following data leakage mitigation, the remaining 39 features are preprocessed to ensure numerical stability and compatibility with machine learning algorithms.

1) *Categorical Feature Encoding*: The dataset contains four categorical features:

- IPV4_SRC_ADDR – Source IP address (1,546 unique values in training set)
- IPV4_DST_ADDR – Destination IP address (1,366 unique values in training set)
- PROTOCOL – Transport protocol (5 unique values: TCP, UDP, ICMP, etc.)
- L7_PROTO – Application-layer protocol (44 unique values: HTTP, DNS, SSH, etc.)

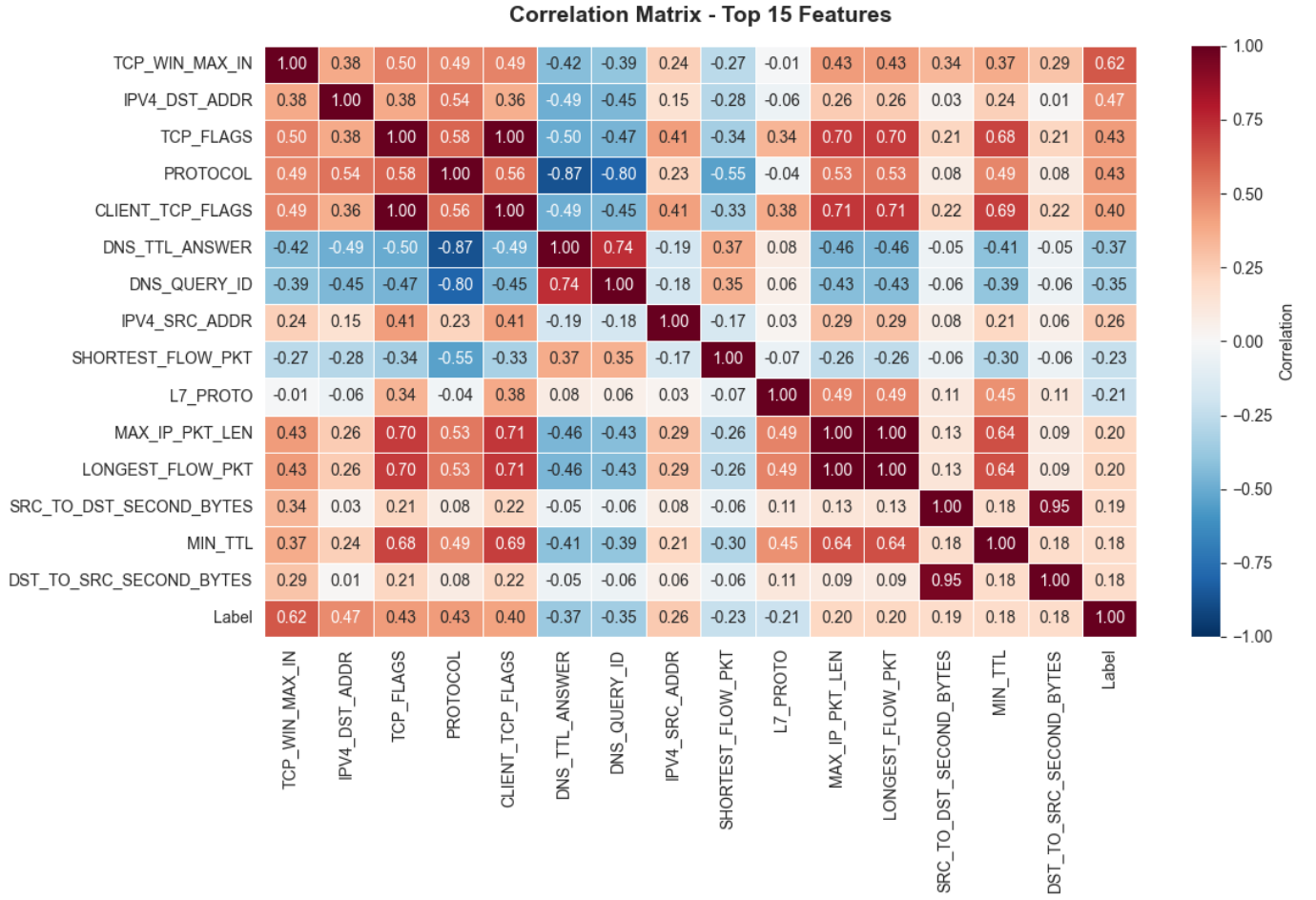


Fig. 4. Correlation matrix of the top 15 features with respect to the binary label.

These categorical features are encoded using label encoding, mapping each unique category to an integer identifier. To handle categories appearing in the test set but unseen during training, a common scenario in network traffic where new IP addresses and protocols emerge continuously, a special value of -1 is assigned to unknown categories, allowing the model to learn a pattern for unseen entities. This strategy avoids runtime failures and preserves a consistent input space during live streaming operation.

a) Encoding of network identifiers (design choice and limitations).: For streaming compatibility and low memory overhead, label encoding was selected as a pragmatic solution to represent high-cardinality identifiers such as `IPV4_SRC_ADDR` and `IPV4_DST_ADDR`. However, this encoding introduces an artificial ordinal relationship between categories (e.g., an IP mapped to 200 is not “twice” an IP mapped to 100), which may be interpreted as meaningful by linear models such as SGD. In this work, these fields are therefore treated primarily as contextual signals rather than as semantically ordered quantities. As future work, more robust alternatives could be explored.

2) Numerical Feature Normalization: The 35 numerical features, including byte counts, packet counts, TCP flags, and throughput metrics, exhibit widely varying scales. To prevent features with large magnitudes from dominating the learning process and to accelerate convergence of gradient-based optimization, standardization is applied using the Standard Scaler transformation:

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

where μ and σ are the mean and standard deviation computed on the training set and applied consistently to test data.

3) Missing Value Imputation: Although the dataset is complete and contains no missing values, a median imputation strategy is applied as a precautionary measure. Median imputation is preferred over mean imputation due to its robustness against outliers, which are common in network traffic data (e.g., large flows or packet bursts during attacks).

Following these preprocessing steps, the final feature matrix contains 39 dimensions: 35 normalized numerical features and 4 integer-encoded categorical features.

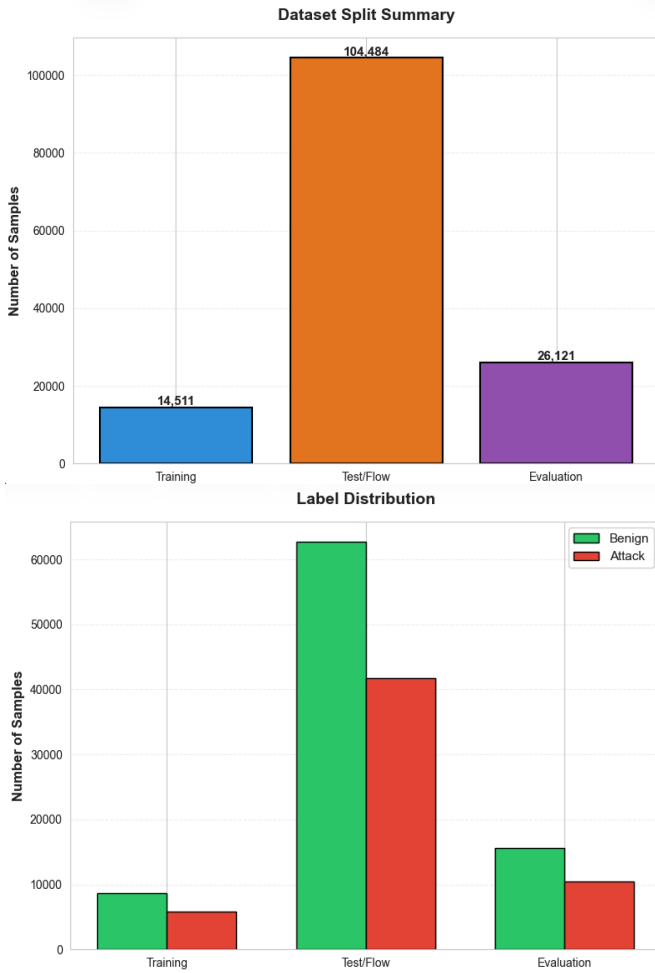


Fig. 5. Dataset split summary and label distributions across training, streaming test, and evaluation sets.

H. Dataset Partitioning

The final dataset is split into three disjoint subsets, as summarized in Figure 5.

A small training set (10%) is used to initialize the incremental binary classifier. The largest portion of the data is reserved for streaming simulation, emulating real-time packet arrival. An additional evaluation set is held out to validate candidate models during online retraining, enabling safe model promotion without interrupting production inference.

This partitioning strategy reflects the operational constraints of an online IDS, where only limited labeled data may be available initially and models must improve progressively as new traffic is observed.

I. Model Training and Selection

This subsection describes the training procedure and selection criteria for both the binary intrusion detection model and the multi-class attack type classifier. Multiple machine learning algorithms were evaluated to balance detection performance, robustness to class imbalance, and suitability for real-time incremental learning.

1) *Binary Classification Models*: For the binary task of distinguishing benign from malicious traffic, several supervised learning algorithms were trained and compared, including Logistic Regression, Support Vector Machines (SVM), k-Nearest Neighbors (KNN), Decision Trees, Random Forests, AdaBoost, Gradient Boosting, and Stochastic Gradient Descent (SGD).

Figure 6 presents a comparison of these models in terms of accuracy, precision, recall, and F1-score, while Figure 7 summarizes their performance using the MCC.

Tree-based ensemble methods, such as Gradient Boosting and Random Forests, achieved the highest overall performance, with MCC values close to 0.97. However, these models operate in a batch-learning regime and cannot be efficiently updated online once deployed.

Despite achieving a lower initial MCC (0.64), the SGD classifier was selected for binary intrusion detection due to its support for incremental learning via the `partial_fit()` interface. This capability is critical for real-world intrusion detection systems, where traffic patterns evolve continuously and models must adapt to concept drift without full retraining. The selected model thus prioritizes long-term adaptability and operational feasibility over static offline performance.

The confusion matrix of the SGD classifier, shown in Figure 8, highlights the trade-off between false positives and false negatives inherent to this choice.

While the false negative rate remains non-negligible at initialization, the model is designed to improve over time through online updates, making it suitable for continuous deployment scenarios.

2) *Multi-Class Attack Type Classification*: For the multi-class task of identifying the specific attack category, a separate set of models was evaluated, including Logistic Regression, Decision Trees, Gradient Boosting, Random Forests, and SGD classifiers. The corresponding performance comparison is shown in Figure 9.

Although incremental learning would also be desirable for attack type classification in a live network, the dataset used in this study is static and fully labeled. Under these conditions, the Random Forest classifier achieved the best overall performance, with high precision, recall, F1-score, and an MCC of 0.976. As a result, it was selected as the attack type classifier.

Figure 10 shows the confusion matrix for the Random Forest model. Most attack categories are classified with near-perfect accuracy.

J. Evaluation Metrics

To provide a comprehensive and reliable evaluation, multiple performance metrics are used throughout the study. Relying solely on accuracy is insufficient due to class imbalance and asymmetric error costs in security applications.

a) *Accuracy*: Accuracy measures the proportion of correctly classified samples. Although intuitive, it can be misleading in imbalanced datasets, where a model predicting only the majority class may still achieve high accuracy.

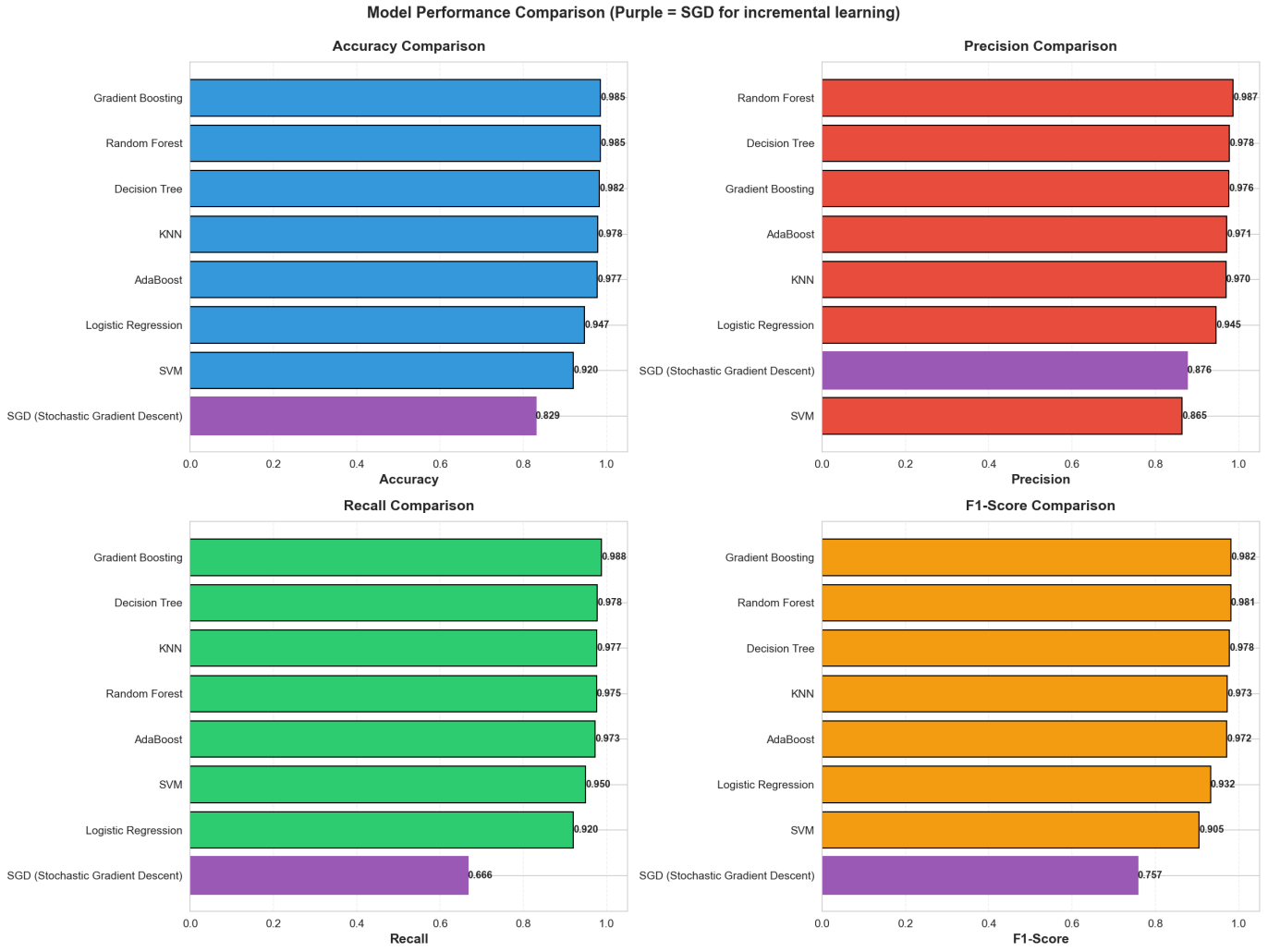


Fig. 6. Binary classification performance comparison across multiple metrics. The SGD model is highlighted due to its incremental learning capability.

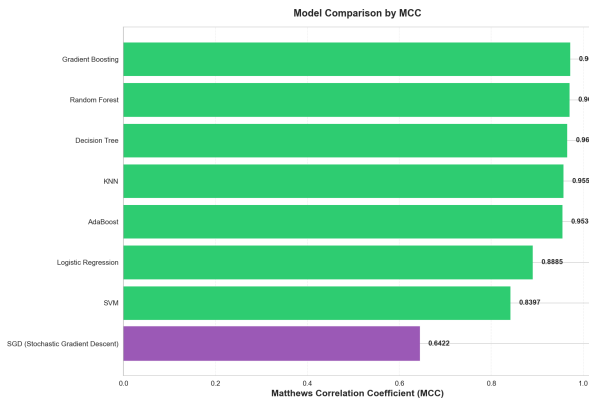


Fig. 7. Binary classification model comparison using Matthews Correlation Coefficient (MCC).

b) Precision: Precision quantifies the proportion of predicted attacks that are true attacks. High precision is essential to limit false positives, which may otherwise overwhelm

security analysts with unnecessary alerts.

c) Recall: Recall measures the proportion of actual attacks that are correctly detected. In intrusion detection, low recall corresponds to missed attacks, which represent a severe security risk.

d) F1-Score: The F1-score is the harmonic mean of precision and recall, providing a balanced summary of detection quality when both false positives and false negatives are important.

e) Matthews Correlation Coefficient (MCC): The MCC is used as the primary evaluation metric, as it accounts for all elements of the confusion matrix (true positives, true negatives, false positives, and false negatives) and remains robust under class imbalance. It is defined as:

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

MCC values range from -1 (total disagreement) to $+1$ (perfect prediction), with 0 indicating random performance. In the context of intrusion detection, MCC provides a more faithful

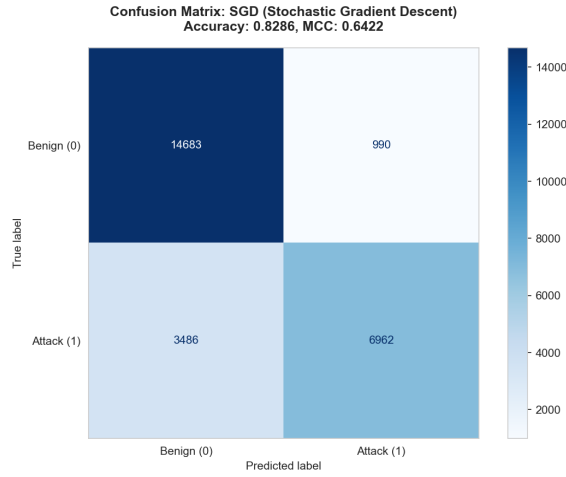


Fig. 8. Confusion matrix of the SGD binary classifier.

representation of real-world effectiveness than accuracy or F1-score alone.

IV. IMPLEMENTATION

This section describes the implementation of the proposed IDS, focusing on the overall software architecture, the main system components, and the mechanisms that enable real-time detection and incremental learning. The implementation emphasizes modularity, robustness, and alignment with realistic deployment constraints.

A. System Architecture Overview

The overall architecture of the system is illustrated in Figure 11. It follows a modular, pipeline-based design in which network flows are processed sequentially, enabling near real-time intrusion detection while supporting continuous model adaptation.

As shown in Figure 11, the system is organized into five main layers: (i) *data ingestion*, implemented by a packet stream simulator that replays the raw test split as an event stream; (ii) *online preprocessing*, where incoming raw flow records are transformed into model-ready features; (iii) *online inference*, where a production SGD classifier performs low-latency binary detection (benign vs. attack), and suspicious flows are forwarded to a Random Forest multi-class classifier for attack family attribution; (iv) *continual learning*, implemented through a shadow-model strategy in which labeled feedback is accumulated in a retraining buffer, used to update a candidate SGD model, and validated on a fixed evaluation set before any promotion to production; and (v) *observability*, comprising a metrics/logging subsystem and a real-time dashboard for monitoring detections, alerts, and retraining outcomes. This separation of concerns improves scalability and allows individual components to evolve independently.

The system is implemented in Python 3.9.6 and relies on scikit-learn for the machine learning models and preprocessing primitives, pandas and NumPy for efficient data handling and

numerical operations, and joblib for efficient model serialization and parallel computation. Streamlit is used to implement the interactive front-end, while visual analytics are rendered with Plotly, enabling interactive time-series and distribution plots directly in the dashboard.

The software is organized following a modular package structure that directly reflects the architectural layers described above. The core implementation resides in the `src/` directory, structured into five main subpackages: `simulator/` (packet stream simulation), `preprocessing/` (online feature transformation), `detector/` (binary and multi-class inference with shadow-model logic), `metrics/` (observability and logging), and `dashboard/` (real-time visualization). The entry point is `run_detection_system.py`, which orchestrates all components through the `IntrusionDetectionSystem` class. Preprocessing artifacts (scalers, encoders, imputers) and trained models are stored separately in `data/preprocessing_artifacts/` and `models/`, enabling offline training and real-time deployment decoupling. This separation ensures modularity, reproducibility, and allows individual components to be developed, tested, and evolved independently.

B. Packet Stream Simulation

To emulate real-time network conditions, a packet stream simulator replays raw NetFlow records from the test split as an event stream. Crucially, the test split is stored in its raw form (i.e., before scaling or encoding) to simulate a realistic deployment scenario in which the system receives unprocessed network telemetry and must apply all transformations online.

Instead of processing the dataset in a static batch mode, samples are emitted sequentially at a configurable rate. This design enables evaluation under conditions that better approximate live deployment, including ordering effects and temporal dynamics.

Each emitted sample contains the raw feature fields and the ground-truth label. The label is used exclusively for evaluation and controlled retraining decisions, and is never exposed to the online inference path.

C. Online Feature Preprocessing

The architecture includes an explicit preprocessing module responsible for transforming raw flow records into the exact feature representation expected by the trained models. This module mirrors the transformations applied during offline training, ensuring consistency between training and deployment.

Given a raw packet or flow, the preprocessing pipeline removes non-deployable and post-hoc features together with label metadata to prevent data leakage. The remaining features are split into numerical and categorical subsets. Numerical features are imputed and normalized, while categorical features are encoded using fixed mappings learned during training; unseen categories at runtime are mapped to a fallback value to ensure robustness. Non-informative features are discarded,

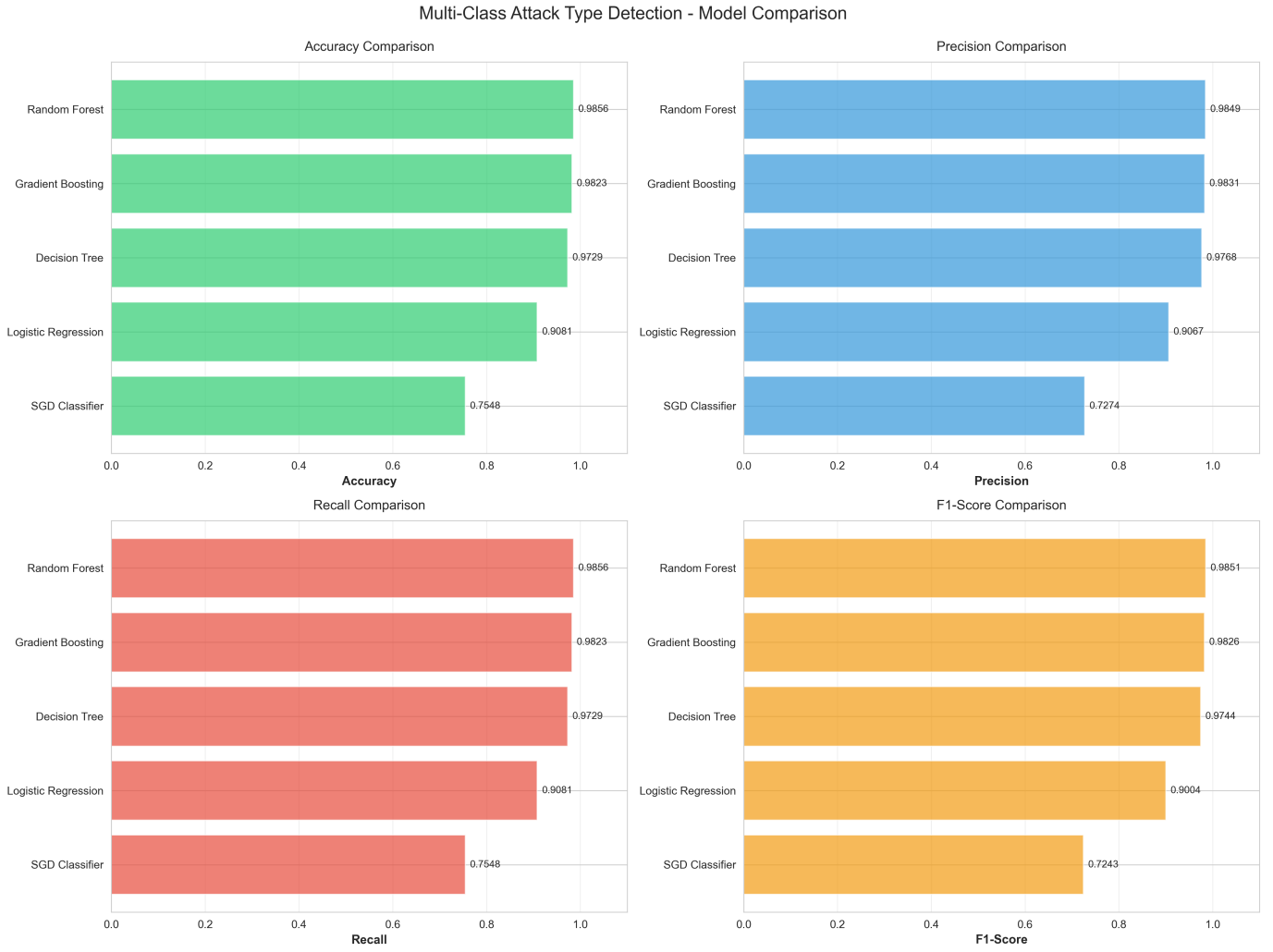


Fig. 9. Multi-class attack type detection performance comparison across multiple metrics.

and the final feature vector is aligned to the training-time configuration by enforcing a fixed ordering and inserting missing features with default values. This guarantees a consistent and well-defined input representation for all downstream inference and learning components.

All preprocessing artifacts are serialized and loaded at runtime, allowing the online pipeline to remain fully consistent with the offline training configuration.

D. Intrusion Detection Engine

The intrusion detection engine is the core runtime component, orchestrating binary inference, attack attribution, feedback collection, and shadow-model updates.

1) Binary Detection Module: The first stage is an SGD-based binary classifier designed for low-latency detection. Incoming raw samples are first transformed by the online preprocessing module and then passed to the production binary model, which outputs a benign or attack prediction.

This module is intentionally lightweight to sustain high-throughput traffic streams with minimal processing overhead.

2) Attack Attribution via Multi-Class Classification: If a flow is classified as malicious by the binary detector, it is forwarded to the multi-class attribution module. This stage relies on a Random Forest model trained offline to assign a specific attack family.

Decoupling binary detection from attack attribution reduces computational cost, since the more expensive multi-class inference is only applied to flows that have already been flagged as suspicious.

E. Continual Learning with a Shadow-Model Strategy

Continual learning is implemented through a shadow-model architecture, where two instances of the binary classifier coexist during execution:

- **Production model**, which serves all incoming traffic and produces live predictions.
- **Candidate (shadow) model**, which is updated in the background using newly collected labeled samples.

New labeled samples are accumulated in a retraining buffer. Once predefined conditions are satisfied (e.g., sufficient buffer

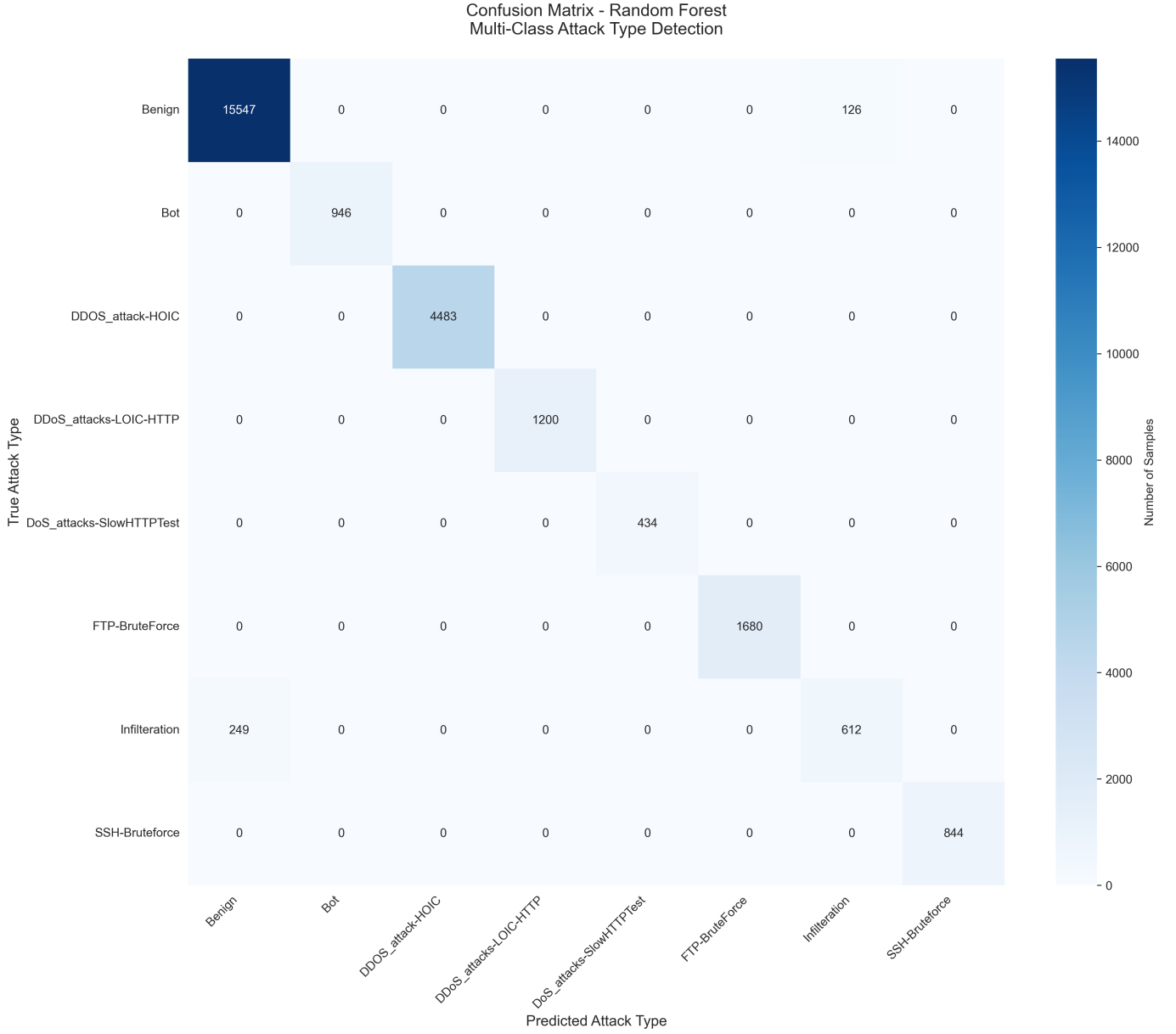


Fig. 10. Confusion matrix of the Random Forest multi-class attack classifier.

size, acceptable class balance, and evidence of performance degradation), the candidate model is updated using mini-batch `partial_fit()` operations.

After each update cycle, the candidate model is evaluated on a fixed validation set. Only if its performance, measured by the MCC, exceeds that of the production model is it promoted. Otherwise, the candidate model is either retained for further updates or reset, preventing regressions and ensuring stable online behavior.

F. Observability: Metrics Tracking and Logging

All predictions, alerts, evaluation results, and retraining decisions are logged during execution. This observability layer supports offline analysis and real-time inspection of system

behavior, enabling reproducibility, debugging, and long-term performance assessment under concept drift.

G. Real-Time Dashboard

A real-time dashboard provides an interactive interface for inspecting detections, alert rates, model performance trends, and retraining history. The dashboard implemented interactive visual analytics, allowing analysts to monitor system behavior during execution and assess how online updates affect performance over time.

V. RESULTS AND EVALUATION

This section presents the experimental results obtained from both offline evaluation and online deployment of the proposed

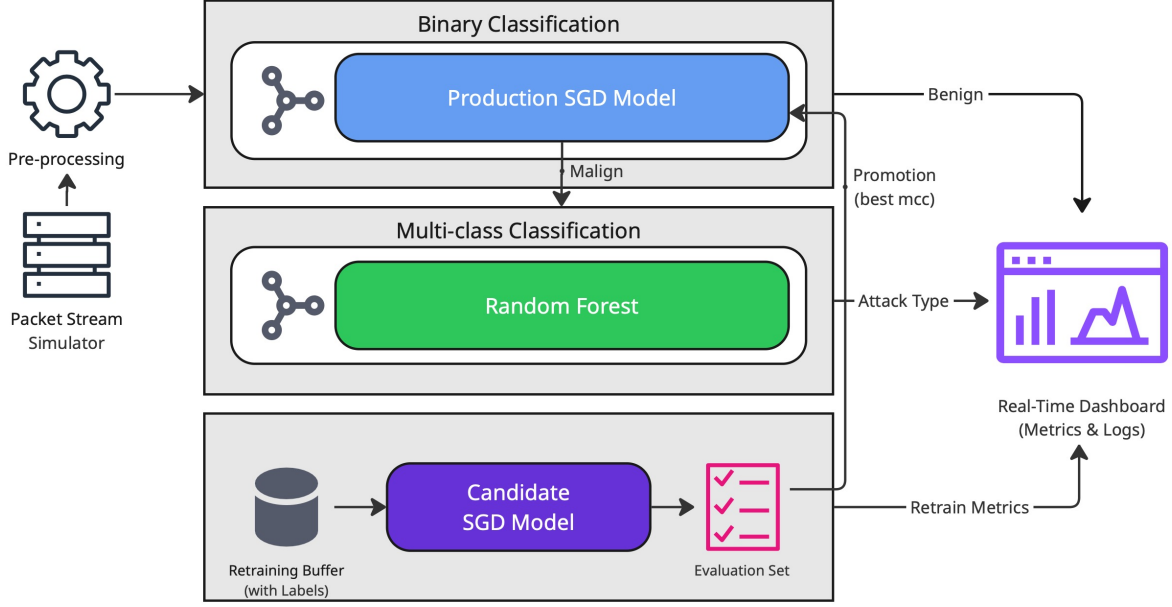


Fig. 11. High-level architecture of the proposed real-time intrusion detection system with incremental learning.

IDS. Particular emphasis is placed on analyzing the behavior of the incremental learning mechanism under streaming conditions and on discussing the associated security trade-offs.

A. Offline Model Performance

Before deployment in a streaming environment, all models were evaluated offline using a held-out test set. As discussed in Section III-I, ensemble-based models such as Gradient Boosting and Random Forest achieved the highest static performance, with MCC values close to 0.97 for binary classification.

However, despite its lower initial performance ($MCC = 0.64$), the SGD-based classifier was selected for binary intrusion detection due to its ability to support incremental updates through `partial_fit()`. This decision prioritizes adaptability over static accuracy, which is critical in environments subject to concept drift.

B. Online Learning Performance

The system was then evaluated in an online setting by processing network flows sequentially and enabling incremental model updates. Figure 12 shows a snapshot of the real-time dashboard during execution.

During the analyzed execution, the system processed 39,579 network flows, of which 17,802 were classified as attacks. A total of 32 retraining events were triggered, reflecting the dynamic nature of the traffic stream and the continuous adaptation of the binary classifier.

C. Model Evolution and Retraining Dynamics

The dashboard highlights the internal dynamics of the incremental learning mechanism. Out of the 32 retraining attempts, only a subset resulted in model promotion, while others were rejected or reset. This behavior confirms the effectiveness of the shadow-model strategy in preventing unvalidated or degraded models from being deployed in production.

The relatively low promotion rate demonstrates that retraining alone does not guarantee performance improvement; instead, only statistically validated gains, measured using the MCC are accepted. This conservative approach favors stability and robustness over aggressive adaptation.

D. Metric Evolution Over Time

Figure 12 also illustrates the evolution of key performance metrics as a function of the number of processed packets. Accuracy shows an upward trend, stabilizing as more data becomes available. More importantly, the MCC curve exhibits a consistent improvement over time, indicating that the classifier is learning to better balance true positives and true negatives as it adapts to the stream.

Precision, recall, and F1-score follow similar trajectories, with short-term fluctuations corresponding to retraining events. These oscillations are expected in online learning settings and reflect the system's responsiveness to newly observed traffic patterns.

E. Security-Oriented Error Analysis

From a security perspective, false negatives are generally more critical than false positives, as undetected attacks may

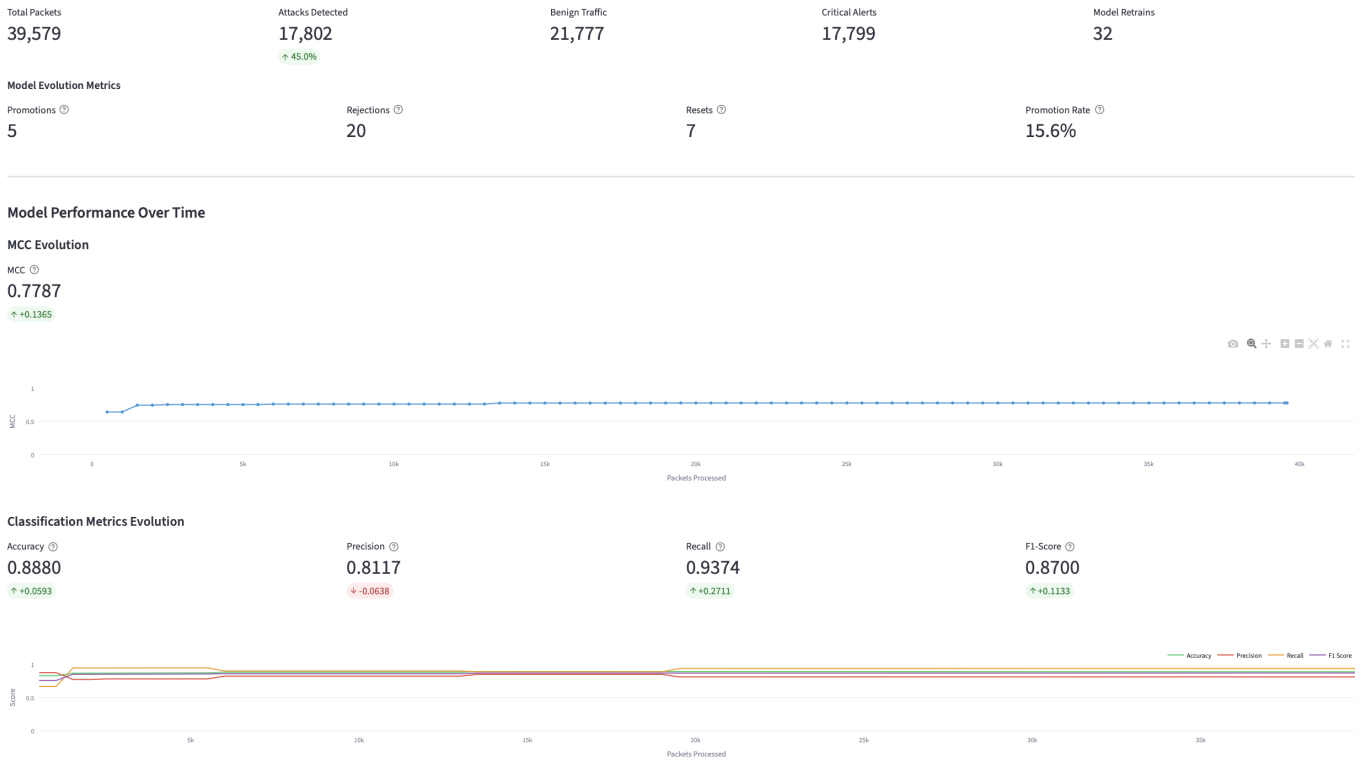


Fig. 12. Real-time dashboard showing online learning performance, retraining events, and metric evolution over time.

lead to system compromise. The initial recall of the SGD classifier is comparatively low; however, online learning significantly improves detection capability over time, as reflected in the rising recall and MCC values.

In this IDS context, the cost asymmetry between these errors is significant: a false negative means malicious traffic is treated as benign and can progress to exploitation, persistence, or lateral movement without triggering mitigation, which directly impacts confidentiality, integrity, and availability. Conversely, a false positive typically results in extra alerts or temporary blocking of legitimate flows, increasing operational overhead and potentially affecting user experience, but without immediately compromising the system. For this reason, our evaluation focuses on recall and F1-score (in addition to MCC), and the online adaptation mechanism is designed to progressively reduce false negatives over time, even if it implies a controlled increase in false positives that can be handled operationally.

False positives, while undesirable, are partially mitigated through confidence thresholds and the two-stage detection architecture. By requiring both a binary attack prediction and sufficient confidence before generating alerts, the system limits unnecessary notifications while maintaining sensitivity to genuine threats.

F. Summary of Results

Overall, the results demonstrate that incremental learning enables the proposed system to progressively improve its detection performance in a realistic streaming scenario. While

static models achieve higher initial performance, they lack the adaptability required for long-term deployment. In contrast, the proposed architecture successfully trades short-term performance for sustained robustness, making it suitable for real-world network environments characterized by evolving traffic and attack behaviors.

VI. CONCLUSION AND FUTURE WORK

This work presented the design, implementation, and evaluation of a real-time IDS with incremental learning capabilities. The proposed solution addresses key limitations of traditional IDSs by combining online adaptability, deployment realism, and robust performance monitoring within a modular and extensible architecture.

Using the NF-CICIDS2018-v3 NetFlow dataset [1], we demonstrated that static, batch-trained models can achieve high offline performance but are ill-suited for long-term deployment in non-stationary environments. To overcome this limitation, the system adopts a two-stage detection pipeline in which an incremental binary classifier performs real-time attack detection, while a high-accuracy multi-class classifier identifies the specific attack type. This separation enables both efficiency and precision, reducing unnecessary computational overhead while maintaining detailed situational awareness.

A central contribution of this work is the integration of incremental learning through a shadow-model strategy. By continuously retraining a candidate model in the background and promoting it only when validated improvements are ob-

served, the system ensures stability and prevents performance regressions in production. Experimental results from an online streaming evaluation show that, despite lower initial performance, the incremental classifier progressively improves over time, as evidenced by rising MCC, recall, and F1-score values. These findings highlight the importance of adaptability over static accuracy in realistic network security scenarios.

From a security perspective, the results demonstrate that the proposed architecture effectively balances false positives and false negatives while maintaining responsiveness to evolving traffic patterns. The real-time dashboard further enhances system transparency, enabling continuous monitoring of detections, retraining events, and model health.

A. Future Work

Several directions for future work emerge from this study. First, extending incremental learning to the multi-class attack classifier would further improve adaptability in long-running deployments, particularly in environments where new attack types gradually emerge. Second, incorporating explicit concept drift detection mechanisms could allow retraining decisions to be triggered more selectively and efficiently.

A key extension is to address model interpretability (XAI) as a core requirement. Since many high-performing approaches behave as black-box models, future work should incorporate explanation methods to clarify why traffic is flagged as malicious or benign. This would help determine whether errors stem from the training data or the model, improving debugging, model quality assessment, and operational trust.

Additional improvements include exploring more advanced online learning algorithms, such as adaptive ensemble methods or neural architectures designed for continual learning. From a deployment perspective, integrating the system with live network sensors and evaluating it under real traffic conditions would provide valuable insights into operational performance and scalability.

Finally, future work could investigate adversarial robustness and resilience against evasion techniques, ensuring that the system remains effective even when faced with adaptive attackers. Together, these extensions would further strengthen the practicality and resilience of incremental learning-based intrusion detection systems.

REFERENCES

- [1] Luay, Majed, S. Layeghy, Mohanad, Sarhan, Sayedehfaezeh, Hoseininoorbin, Moustafa, Nour, and M. Portmann, "Nf-cse-cic-ids2018-v3," 2025. [Online]. Available: <https://doi.org/10.48610/ece9b83>
- [2] J. Talpini, F. Sartori, and M. Savi, "Hierarchical Multiclass Continual Learning for Network Intrusion Detection," in *2024 IEEE 10th International Conference on Network Softwarization (NetSoft)*. Saint Louis, MO, USA: IEEE, Jun. 2024, pp. 263–267.
- [3] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, Funchal, Portugal, Jan. 2018, pp. 108–116.
- [4] F. Cerasuolo, G. Bovenzi, A. Montieri, and A. Pescapè, "Class Incremental Learning for Network-Agnostic Intrusion Detection Systems," in *2025 IEEE 9th Forum on Research and Technologies for Society and Industry (RTSI)*. Tunis, Tunisia: IEEE, Aug. 2025, pp. 17–22.